PumpScript

Example Programs & Test Report – Part 2

CSE 341 – Concepts of Programming Languages
Gebze Technical University – 22 May 2026

Student Name: Yusuf Eren Nalbant
Student No: 220104004049

Pipeline: Lexer → Parser → Type Checker → Interpreter
Run command: `python3 pumpscript.py <file.pump>`

Overview

This report documents the final Part 2 test set. The same three non-trivial example programs from the implementation are executed end-to-end, and one additional program demonstrates a static type error caught before execution. I also include the five malformed parser/lexer tests from Part 1 in compact form to show that syntax error reporting still works.

File	Result	Stage	Main features exercised
`valid1.pump`	PASS	Interpreter	routine, parameter, exercise, rest
`valid2.pump`	PASS	Interpreter	several exercises, literals and identifiers
`valid3.pump`	PASS	Interpreter	multiple routines, `if`, `repeat`, rest
`type_error.pump`	TYPE ERROR	Type Checker	wrong type for `weight`

1 Valid Program 1 – Chest Day Routine

Source focus. `valid1.pump` defines `routine chest_day(sets: int)`. Inside the routine there are two exercise records, `bench_press` and `chest_fly`, separated by `rest 60 seconds`. The program ends with `chest_day(4)`.

```
routine chest_day(sets: int) {
    exercise bench_press { sets: sets reps: 10 weight: 80.0 group: "chest" completed: false }
    rest 60 seconds
    exercise chest_fly { sets: sets reps: 12 weight: 20.0 group: "chest" completed: false }
}
chest_day(4)
```

Actual output.

```
Exercise: bench_press | sets: 4 | reps: 10 | weight: 80.0 kg | group: chest | completed: no
--- Rest: 60 seconds ---
Exercise: chest_fly | sets: 4 | reps: 12 | weight: 20.0 kg | group: chest | completed: no
```

Discussion. This program demonstrates parameter passing and static scoping: the parameter `sets` is bound to `4` when `chest_day(4)` is called, and both exercise blocks resolve that name inside the routine body. It also shows the domain-specific `rest` statement producing visible workout output. The boolean field `completed: false` is printed as `no`, which is an interpreter formatting decision.

2 Valid Program 2 – Leg Day Routine

Source focus. `valid2.pump` defines `leg_day(sets: int)` with three exercise records and two rest periods. The `squat` and `leg_press` records use the routine parameter, while `lunges` uses a literal `sets: 3`.

```
routine leg_day(sets: int) {
    exercise squat { sets: sets reps: 8 weight: 100.0 group: "legs" completed: false }
    rest 90 seconds
    exercise lunges { sets: 3 reps: 12 weight: 0.0 group: "legs" completed: false }
    rest 60 seconds
    exercise leg_press { sets: sets reps: 10 weight: 120.0 group: "legs" completed: false }
}
leg_day(4)
```

Actual output.

```
Exercise: squat | sets: 4 | reps: 8 | weight: 100.0 kg | group: legs | completed: no
--- Rest: 90 seconds ---
Exercise: lunges | sets: 3 | reps: 12 | weight: 0.0 kg | group: legs | completed: no
--- Rest: 60 seconds ---
Exercise: leg_press | sets: 4 | reps: 10 | weight: 120.0 kg | group: legs | completed: no
```

Discussion. This program demonstrates that exercise fields may be either literals or parameter references. It also shows that the structured exercise schema is checked repeatedly for multiple exercise blocks. Because every exercise has the same required field set, the type checker can reject missing, duplicate, unknown, or wrongly typed fields before interpretation.

3 Valid Program 3 – Full Body Routine with Control Structures

Source focus. `valid3.pump` contains three routines: `upper_body`, `lower_body`, and `core`. The top-level statements intentionally use both control structures required by the design: an `if true { ... } else { ... }` statement and a `repeat 1 times { ... }` statement.

```
if true { upper_body(4) } else { core(1) }
rest 120 seconds
repeat 1 times { lower_body(3) }
rest 90 seconds
core(3)
```

Actual output.

```
Exercise: pull_up | sets: 4 | reps: 8 | weight: 0.0 kg | group: back | completed: no
--- Rest: 90 seconds ---
Exercise: shoulder_press | sets: 4 | reps: 10 | weight: 40.0 kg | group: shoulders | completed: no
--- Rest: 120 seconds ---
Exercise: deadlift | sets: 3 | reps: 5 | weight: 140.0 kg | group: back | completed: no
--- Rest: 120 seconds ---
Exercise: calf_raise | sets: 3 | reps: 15 | weight: 60.0 kg | group: legs | completed: no
--- Rest: 90 seconds ---
Exercise: plank | sets: 3 | reps: 1 | weight: 0.0 kg | group: core | completed: no
--- Rest: 45 seconds ---
Exercise: crunch | sets: 3 | reps: 20 | weight: 0.0 kg | group: core | completed: no
```

Discussion. This is the most complete example. The `if` condition is a boolean literal, so the then-branch calls `upper_body(4)` and the else-branch is skipped. The `repeat 1 times` block executes `lower_body(3)` exactly once, demonstrating the repeat semantics without producing excessive output. The final `core(3)` call shows that multiple routine calls can be sequenced at the top level.

4 Static Type Error Test

Source focus. `type_error.pump` assigns a string to the `weight` field, which is declared as `float` in the exercise schema.

```
exercise squat {
    sets: 8
    reps: 8
    weight: "heavy"
    group: "legs"
    completed: false
}
```

Actual error message.

```
$ python3 pumpscript.py examples/type_error.pump
[Type Error] Line 6: In exercise 'squat': field 'weight' expects float but got string
```

This error is caught before interpretation, so no partial workout output is produced. That behavior is consistent with the D1 claim that PumpScript is strongly typed.

5 Malformed Program Regression Tests

File	Actual error message
error1.pump	[Parse Error] Line 10: Expected '}' but found "
error2.pump	[Lexer Error] Line 4: Unexpected character '@'
error3.pump	[Parse Error] Line 2: Expected ':' but found ')'
error4.pump	[Lexer Error] Line 7: Unterminated string literal - missing closing '"'
error5.pump	[Parse Error] Line 12: Expected '(' but found '4'

Conclusion

The final implementation satisfies the Part 2 pipeline: all valid examples pass through lexing, parsing, type checking, and interpretation; malformed syntax is rejected with line-numbered errors; and the static type checker rejects an invalid exercise field before execution.